

Reviewer Pack — Minimal Tropical Hodge Structure Rank Bound for Resolution P...

Entry 50acebb6334e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-09 22:29:14 UTC

Minimal Tropical Hodge Structure Rank Bound for Resolution Proof Width

Entry ID: 50acebb6334e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-09 22:29:14 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry - Hodge Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every Tseitin formula φ_G with n variables, the minimal tropical Hodge structure rank ($\text{th}(\varphi_G)$) of its associated matrix ring is linearly correlated with its resolution proof width $w(\varphi_G)$, such that $\text{th}(\varphi_G) = \Theta(w(\varphi_G))$.

Rationale (proposer's reasoning):

The connection between tropical geometry and algebraic structures such as Hodge theory has not been extensively explored in complexity theory, yet it provides a framework for studying the geometric properties of complex objects. This conjecture proposes that the rank of the tropical Hodge structure can serve as an invariant to bound the resolution proof width, potentially revealing new insights into the complexity of Boolean satisfiability problems.

Taxonomy category: TROPICAL_HODGE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ea83d63a169f5a7a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every Tseitin formula φ_G , if the correlation coefficient between $\text{th}(\varphi_G)$ and $w(\varphi_G)$ is above 0.8 with a p-value ≤ 0.05 over 30 random seeds, then support for the conjecture is provided; otherwise, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND Hodge theory AND resolution proof complexity
- resolution proof width AND tropical Hodge structure rank bound - Tseitin formula AND
minimal tropical Hodge structure rank AND resolution proof

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/math/0403015v1] Amoebas of algebraic varieties and tropical geometry - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/astro-ph/0208243v7] Measurement of the Flux of Ultrahigh Energy Cosmic Rays from Monocular Observations by the High Resolution Fly’s Eye Exp - [http://arxiv.org/abs/astro-ph/0407622v3] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly’s Eye - [http://arxiv.org/abs/1901.10330v2] Canonisation and Definability for Graphs of Bounded Rank Width - [http://arxiv.org/abs/2007.07826v1] Hodge theory for tropical varieties - [http://arxiv.org/abs/1506.04846v1] Analytification and Tropicalization over non-archimedean fields

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []

        # Generate clauses for each variable
        for var in variables:
            clauses.append([var])
            clauses.append([-var])

        # Generate clauses for each pair of variables
        for i in range(n):
            for j in range(i + 1, n):
                clauses.append([variables[i], variables[j]])
                clauses.append([variables[i], -variables[j]])
                clauses.append([-variables[i], variables[j]])
                clauses.append([-variables[i], -variables[j]])
```

```

    # Generate the final clause
    final_clause = []
    for var in variables:
        final_clause.append(var)
    clauses.append(final_clause)

    return clauses

def dpll(clauses, assignment):
    if not clauses:
        return True
    unit_clauses = [c[0] for c in clauses if len(c) == 1]
    if unit_clauses:
        literal = unit_clauses[0]
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment):
            return True
        new_assignment[literal] = False
        if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment):
            return True
        else:
            return False

    pure_literals = []
    for literal in assignment.keys():
        positive_count = sum(1 for clause in clauses if literal in clause)
        negative_count = sum(1 for clause in clauses if -literal in clause)
        if positive_count == 0 and literal not in pure_literals:
            pure_literals.append(literal)
        elif negative_count == 0 and -literal not in pure_literals:
            pure_literals.append(-literal)

    if pure_literals:
        literal = pure_literals[0]
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment):
            return True
        else:
            return False

    literal = random.choice(list(assignment.keys()))
    new_assignment = assignment.copy()
    new_assignment[literal] = True
    if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment):
        return True
    else:
        new_assignment[literal] = False
        if dpll([c for c in clauses if literal not in c and -literal not in c], new_assignment):
            return True
        else:
            return False

def resolution(clauses):

```

```

while True:
    unit_clauses = [c[0] for c in clauses if len(c) == 1]
    if not unit_clauses:
        break
    literal = unit_clauses[0]
    new_clauses = []
    for clause in clauses:
        if literal in clause:
            continue
        elif -literal in clause:
            new_clauses.extend([c for c in clauses if c != clause and -c != clause])
            break
        else:
            new_clauses.append(clause)
    clauses = new_clauses

def tropical_hodge_structure_rank(phi_G):
    n = len(phi_G[0])
    A = [[-math.inf] * n for _ in range(n)]

    for clause in phi_G:
        for i, var in enumerate(clause):
            if var > 0:
                A[i][var - 1] = max(A[i][var - 1], 1)
            else:
                A[i][-var - 1] = max(A[i][-var - 1], 1)

    for i in range(n):
        for j in range(i + 1, n):
            A[i][j] = max(A[i][j], A[j][i])
            A[j][i] = max(A[j][i], A[i][j])

    rank = 0
    for row in A:
        if any(x != -math.inf for x in row):
            rank += 1

    return rank

def resolution_width(phi_G):
    n = len(phi_G[0])
    assignment = {f'x{i}': False for i in range(1, n+1)}
    width = 0
    while not dpll(phi_G, assignment):
        new_clause = []
        for i in range(n):
            if not assignment[f'x{i+1}']:
                new_clause.append(f'x{i+1}')
            else:
                new_clause.append(-f'x{i+1}')
        phi_G.append(new_clause)
        width += 1
    return width

n = random.randint(5, 40)
phi_G = generate_tseitin_formula(n)

```

```

th_phi_G = tropical_hodge_structure_rank(phi_G)
w_phi_G = resolution_width(phi_G)

return {
    "metric_name": "resolution_width",
    "metric_value": w_phi_G,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": th_phi_G == w_phi_G,
    "counterexample": "" if th_phi_G == w_phi_G else f"th( $\phi_G$ )={th_phi_G}, w( $\phi_G$ )={w_phi_G}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"th( $\phi_G$ ) != w( $\phi_G$ )\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_14c6d5f3.py", line 167, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_14c6d5f3.py", line 148, in run_trial
    phi_G = generate_tseitin_formula(n)
            ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_14c6d5f3.py", line 28, in generate_tseitin_formula
    clauses.append([-var])
                   ~~~^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required calculations to determine the correlation coefficient and p-value. | next: Review the code for errors causing the crash and attempt to run the test again.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18073
2	preregistration	ollama_remote	glm4:latest	0	0	14064
3	novelty	ollama_remote	glm4:latest	0	0	13315
4	novelty	ollama_remote	glm4:latest	0	0	10146
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24354
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13403
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10298
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18834
9	judge	ollama_remote	glm4:latest	0	0	15832

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 138319 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/50acebb6334e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/50acebb6334e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/50acebb6334e.tar.gz (if generated)